CSCI 532 - Algorithms

Ben Heinze

April 9, 2026

1 Tutorial for ORG Mode:

Let A be an $n \times n$ matrix. The determinant satisfies:

$$111 \det(AB) = \det(A) \cdot \det(B)$$

1.1 Gaussian Elimination (Pseudo-code)

```
INPUT: augmented matrix [A | b]
FOR each pivot row r:
  FOR each row below r:
    factor ← A[i][r] / A[r][r]
    A[i] ← A[i] - factor * A[r]
RETURN reduced matrix
```

2 Integer Linear Programming

Vertex cover: given graph $G = (V, E)$ find the smallest V' $\subset$ V such that each edge in E contains an end point in V':

- $x_i \in \{0, 1\}$ = indicates if vertex i is selected.
- Objective: min $\sum_i x_i$
- subject to:
  - $x_i + x_j \leq 1$, for each edge $e = (i, j)$

Solving integer linear programs (ILPs) is NP-Hard (in general).

ILP vs LP: Since the PL has more options to reduce the objective value, $OPT_{LP} \leq OPT_{ILP}$ (for a minimization problem). If the minimum objective value comes from an integer solution, a plain LP solver (e.g., Simplex) will find it. **Go home and fundamentally understand **$OPT_{LP} \leq OPT_{ILP}$ **becaues it may be a test question**

2.1 Why are ILPs hard to solve?

Does the optimal continuous solution –> optimal integer solution?

[insert graph pic here]

The continuous optimal objective function is 41.25. How can we translate that to an ILP solution?

- Closest integer solution?

  - not feasible

- Closest feasible integer solution?
  - objective function = 34, but not optimal
- Closest feasible integer solutionnn on feasible region boundary?
  - objective function = 39 = not optimal
- Actual optimal:
  - objective function = 40

[insert image of other shrinkwrap graph]
Integer feasible region:

- not convex
- local optimum $\neq$ global optimum

Minimum convex hull (integer hull):

- convex
- local optimum = global optimum
- $O(n^{\lfloor \frac{d}{2} \rfloor})$ faces,
  - n = number of points
  - d = number of dimensions

2.2 Solving ILPs

Supposed we have an ILP that is a maximization problem.

1. Relax ILP (turn integers into real values) $x \in \mathbb{Z} \rightarrow x \in \mathbb{R}$
2. Solve relaxed LP. Suppose the optimal objective is 100 with a fractional solution (ig some $x_i = 3.24$) Since the relaxed LP has more optinons to increase the objective value, $OPT_{LP} \leq OPT_{ILP}$
3. Formulate two neew LPs. One with $x_i \geq 4$ and the other one with $x_i \leq 3$ constraints. New you have two LP subproblems that you can solve easily.
4. Solve subproblem LPs
5. Formulate two new LPs that split the subproblem on some non-integer variable
6. Solve the subproblem LPs.
7. Repeat.

2.2.1 case 1: both subproblems are integers

Main problem (LP relaxation): $x \in \mathbb{R}, obj = 100$
Subproblem 1 result: $x \in \mathbb{Z}, obj = 85$
Subproblem 2 result: $x \in \mathbb{Z}, obj = 90$
If both subproblems are integers ($\in \mathbb{Z}$), just select the highest one (subproblem 2 in this case).

2.2.2 case 2: a subproblem is in the real number set

subproblem 1: result $= x \in \mathbb{R}, obj = 98$

subproblem 2: result $=x \in \mathbb{Z}, obj = 90$

Subproblem 1 isn't guaranteed to be an integer, but now we have two bounds. Worst case, it is 90. Best case, it is 98.

So far, we have:

1. feasible (integer) solution (subproblem 2)
2. Upper and lower bounds on optimal (90, 98).

Branch and bound plan: use these to restrict the search space and identify optimality.

Add two sub-subproblems on subprolbem 1:

Subproblem 1.1: result $= x \in \mathbb{R}, obj = 88$

Subproblem 1.2: result $= x \in \mathbb{R}, obj = 97$

Subproblem 1.1 is dead, beacuse the objective function is already lower than our first integer subproblem 2, where the integer objective was 90. No need to check worse solutions.

If subproblem 1.2 had an objective solution of 85, we terminate both subproblems since we know that subproblem 2's obj = 90 solution is optimal.

2.2.3 Is this process guaranteed to eventually find the optimal integer solution?

Yes, this is essentially brute-force with heuristics. Given enough constraints like $x_i \leq 3$, all variables will be bounded to their optimal intger values. Other techniques (cutting planes, intelligently picking which x_i to split on) can help speed up the process.

2.3 Can we optimally solve this?

Variables: $x_1, x_2 \in \mathbb{Z}$

Objective: $min - \frac{\pi}{\sqrt{2}}x_1 + \frac{\log 2}{e}x_2 + \frac{32}{17}$

Subject to: $x_1 \leq 1; x_2 \leq 1; x_1, x_2 \geq 0$

Can we solve this optimally?

yes..

3 Vertex Cover

Total Unimodularity: A matrix is **totally unimodular** if the determinant of any square submatrix of it is 0, 1, or -1.

- This impies that totally unimodular matrices are composed of 0s 1s or -1s, since single elements are square submatrices.
- This also means that if A is totally unimodular, A^T is too since $det(B) = det(B^T)$

Theorem (Hoffman-Krusal, 1956):

- $A \in \mathbb{R}^{m \times n}$is totally unimodular $\iff$ the feasible region:$\{x \in \mathbb{R}^n \mid Ax \leq b, x \geq 0\}$has integer vertices $\forall b \in \mathbb{Z}^m$.
  - If we can prove that our coefficient constraint matrix is unimodular, we can solve it in polynomial time.

Objective: $max(c^T x)$ Subject to: $Ax \leq b; x \geq 0$

If A is totally unimodular, the ILP can be solved in polynomial time! However, there is no quick and easy way to determine if a matrix is unimodular until Ghoula-Houri's theorem came out:

Theorem (Ghoula-Houri, 1962):

- $A \in \mathbb{R}^{m \times n}$ is unimodular $\iff$ for every subset of rows R, there is a partition $R = R_1 \bigcup R_2$ such that for every column j, $\sum_{i \in R_1} A_{ij} - \sum_{i \in R_2} A_{ij} \in \{-1, 0, 1\}$

These two theorems are linked together by the unimodularity. We can show if a problem has the Ghoula-houri property, then it has an integer solution —

4 Vertex cover:

Vertex cover ILP:

- $x_i \in \{0, 1\}$ = indicates if vertex i is selected
- Objective: $max(c^T x)$
- Subject to: $Ax \leq b; x \geq 0$

Goula-Houri Theorem:

- Objective: $min \sum_i x_i$
- subject to: $x_i + x_j \geq 1$, for each edge $e = (i, j)$

We can create a matrix A from the vertex cover graph such. Now that we have a matrix, we need to transpose $A \to A^T$ in order to use the Ghoula-Houri theorem.

We can use an Label alternating generations of vertices into two sets B and G that would guarantee that every edge has two different colored endpoints

To turn our problem to work with the GoulaHouri theorem, we must change $\leq$ to $\geq$ in the contraints. Look this up in the lecture for details

Vertex cover ILP:

- $x_i \in \{0, 1\}$ = indicates if vertex i is selected
- Objective: $max(c^T x)$
- Subject to: $Ax \geq b; x \geq 0$

Assume we have a matrix A that has edges on the rows, and vertices on the columns. This has a guaranteed row structure, but we need guaranteed column structure (since edges are necessary for a vertex to exist), which is why we transform $A \to A^T$. Now we have vertices on the rows and edges on the columns instead.So we can use Ghouila-Houris theorem:

$$A \in \mathbb{R}^{m \times n} \text{ is unimodular } \iff \text{ for every subset of rows R, there is a partition } R = R_1 \bigcup R_2$$

such that for every column j, $\sum_{i \in R_1} A_{ij} - \sum_{i \in R_2} A_{ij} \in \{-1, 0, 1\}$

We will come up with a partition that works for every single tree. How can we partition the tree such that we only get two unique subsets such that vertices that share an edge are in different subsets

Label alternating generations of vertices into two sets B and G. Each level of the tree is either B or G, which guarantees that no edges vertices sharing an edge are always in different sets. If that weren't true, then an edge would have to span generations (creates a cycle, which doesnt exist in this tree). Consider a subset of rows of A^T. Partition R into $R_1 = R \cap B$ and $R \cap G$

For any column of A^T, which corresponds to an edge e

$$\sum_{v \in R_1} A_{ve} \in \{0,1\} \text{ and } \sum_{v \in R_r} A \in \{0,1\} \rightarrow \sum_{v \in R_1} A_{ve} - \sum_{v \in R_2} A_{ve} \in \{-1,0,1\}$$

Therefore Vertex Cover ILP will be solved in polynomial time when the graph is a tree. This solution will work on any bipartite graph.

5 Vertex Cover (General Graph)

Goal: Show why this specific graph does not have the Ghouila-Houri property.

Where is the contradition with Ghouila-Houri?

x_1 and x_2 have to be on different sides of the partition. There are only two possible partitions:

- $R_1 = \{x_1, x_4\}, R_2 = \{x_x\}$, which column e_2 sums to 2; not totally unimodular
- $R_1 = \{x_1\}, R_2 = \{x_2, x_4\}$, which column e_4 sums to -2; not totally unimodular

No such partition exists, so A^T and A are not totally unimodualar. **Total unimodularity is not the only route to an integer-vertexed feasible region.**

5.1 Non-totally unimodular ILP that can have integer feasible region.

To prove that total unimodularity is not the only route to an integer-vertexed feasible region, he showed an example of an ILP, showed the matrix A is not totally unimodular, then plotted the feasible regions of the ILP that proved the answer was integer.

5.2 Circuit Value Problem: When the gates are evaluated, is the output true?

If we turned a circuit diagram into an ILP by turning each node into a decision variables that has 1 and 0 as values, we can figure out the solution. Once we relax this problem, the answer is guaranteed to be integer since the inputs are all integers, and the contstraints keep them integers. **This problem does not work well with Total Unimodularity.** He proves this in the slides by converting the graph to a matrix and showed the determinant is -2, which does not fall into the definition of being {-1, 0, 1}. But the solution is solvable by other solutions.